



The Energy University

CSEB5223 – SOFTWARE CONSTRUCTION & METHODS
SEMESTER 2 2025/2026

LAB 10 (ASSIGNMENT 2)

GITHUB REPOSITORY NAME:

cherrySLMS

SECTION: 01BT

PREPARED FOR:

TS. DR. LOO YIM LING

PREPARED BY:

NO.	NAME	ID
1	Nur Sahira Binti Mohd Nizam	BSW01085247
2	Aimi Syazwani Binti Mohd Ezanee	BSW01085441
3	Wan Nur Alisya Sofia Binti Wan Zainudin	BSW01085261

TABLE OF CONTENTS

1.0.0 Use Case and Class Design of Overall SLMS	4
1.1.0 Use Case Design	4
1.2.0 Class Design.....	6
2.0.0 Documentation Formatting.....	8
3.0.0 Coding File Formatting	9
4.0.0 Naming Conventions.....	10
4.1.0 Class Naming	10
4.2.0 Attribute Naming	10
4.3.0 Method Naming	11
4.4.0 Constant Naming	11
4.5.0 Vocabulary Dictionary	12
5.0.0 Function and Methods Modularity	13
5.1.0 Encapsulation Principle	13
5.2.0 Single Responsibility Principle.....	13
5.3.0 Getter and Setter Methods	13
5.4.0 Method Size and Complexity Control	14
5.5.0 Separation of Responsibilities.....	14
5.6.0 Reusability and Maintainability	15
6.0.0 Continuous Integration Rules.....	16
6.1.0 Branching Strategy.....	16
6.2.0 Commit Message Format.....	16
6.3.0 Merge Rules	16
7.0.0 System Main Menu	18
7.0.1 Manage Courses Function.....	18
7.0.2 Manage Students Function.....	18
7.0.3 Manage Enrollments Function	19
7.0.4 Exit System	19
7.0.5 Input Validation.....	20
8.0.0 Course Management Menu	22
8.0.1 Add Course Function	22
8.0.2 Search Course Function	24

8.0.3 Edit Course Function	26
8.0.4 Delete Course Function.....	28
8.0.5 View All Courses Function	30
8.1.0 Analysis of Software Construction Best Practices and Challenges	31
8.1.1 Software Construction Best Practices	31
8.1.2 Challenges.....	32
9.0.0 Student Management Menu.....	33
9.0.1 Add Student Function	33
9.0.2 Search Student Function	35
9.0.3 Edit Student Function	36
9.0.4 Delete Student Function.....	38
9.0.5 View All Students Function	40
9.1.0 Analysis of Software Construction Best Practices and Challenges	40
9.1.1 Software Construction Best Practices	40
9.1.2 Challenges.....	42
10.0.0 Enrollment Management Menu.....	43
10.0.1 Add a Course to a Student Function	43
10.0.2 Add a Student to a Course Function	45
10.0.3 Find a Student's Course by Student ID Function.....	47
10.0.4 List all Courses enrolled by a Student Function	49
10.0.5 Find a Student in a Course by Course Code Function	50
10.0.6 List all Students enrolled in a Course Function	51
10.1.0 Implementation of the Suggestion API	53
10.1.1 Implementation of Caching in Course Management	53
10.1.2 Implementation of Caching in Student Management	55
10.2.0 Analysis of Software Construction Best Practices and Challenges	56
10.2.1 Software Construction Best Practices	56
10.2.2 Challenges.....	58

1.0.0 Use Case and Class Design of Overall SLMS

1.1.0 Use Case Design

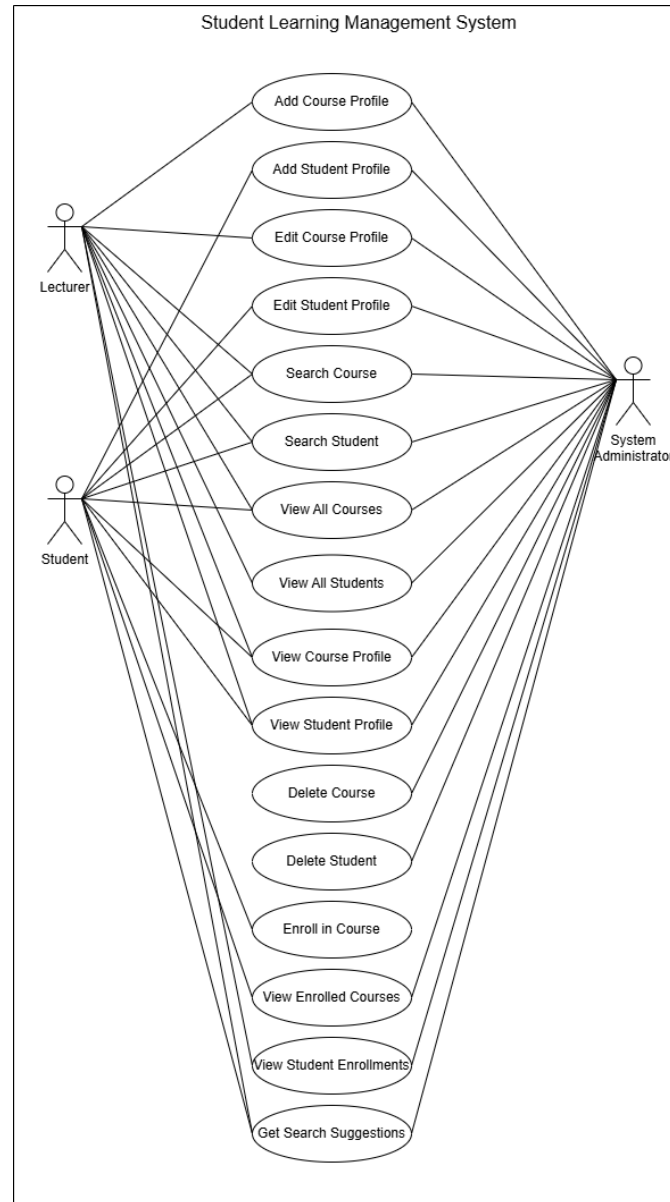


Figure 1.1 Student Learning Management System's (SLMS) Use Case Diagram

- The use case diagram illustrates the main functionalities of the Student Learning Management System (SLMS) and the actors that interact with the system.
- Three actors are involved: Lecturer, Student and System Administrator, each with different access rights based on their roles.

- The Lecturer can add, edit, search, and view course and student profiles to support academic management activities. In addition, the Lecturer can also view student enrollments to monitor students registered in different courses.
- The Student can search and view course information, view their own student profile, and enroll in courses offered by the system. Students are also able to view the courses they have enrolled in.
- The System Administrator has full access to the system, including adding, editing, searching, viewing, and deleting both course and student profiles. The administrator is also responsible for managing enrollments and can view student enrollment details across all courses.
- To improve usability, the system includes a search enhancement feature where users can receive search suggestions while searching for courses or students, helping to speed up and improve the accuracy of search results.

1.2.0 Class Design

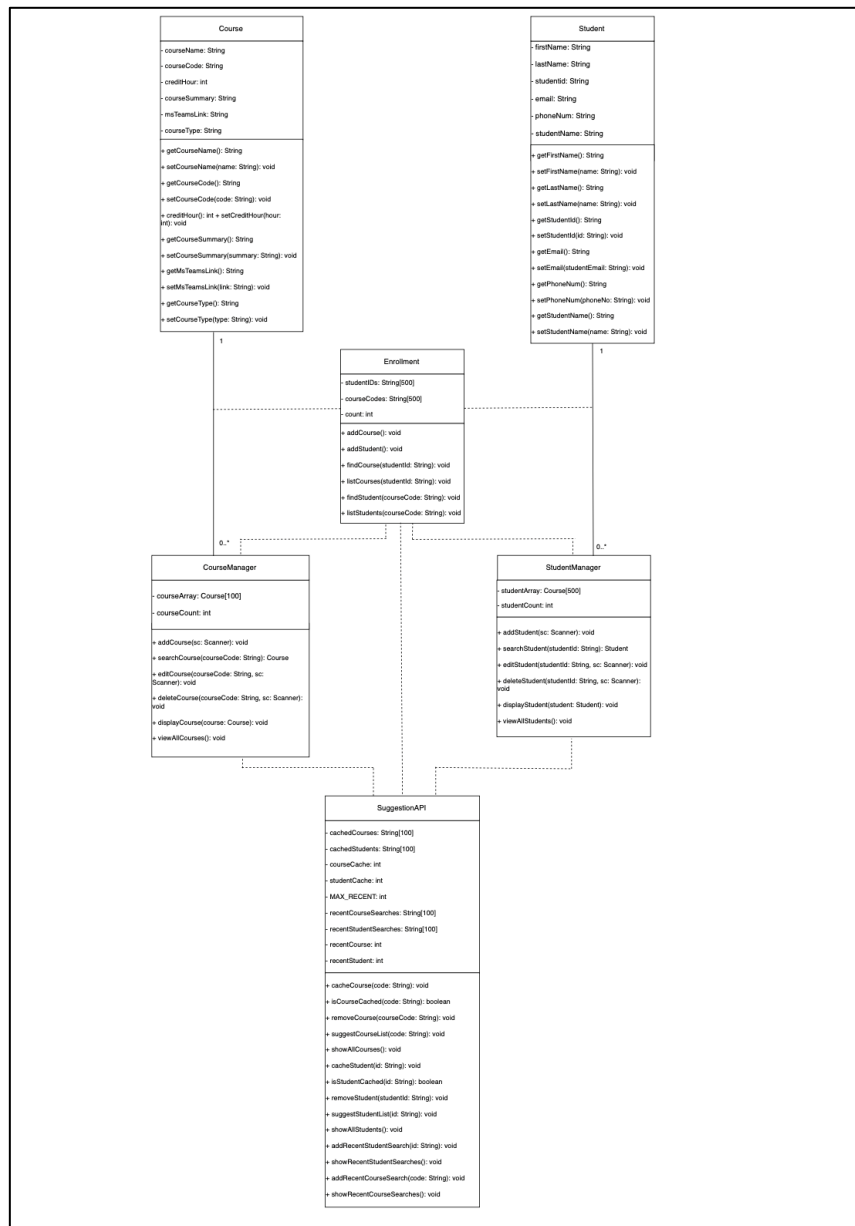


Figure 1.2: Student Learning Management System's (SLMS) Class Diagram

- The class diagram shows the main entities of the Student Learning Management System (SLMS) and how they are related.
- The Course class represents course details such as course name, code, credit hour, summary, MS Teams link, and course type. The Student class represents student

information including first name, last name, student ID, email, phone number, and student name.

- The Enrollment class models the relationship between students and courses. A student can be enrolled in multiple courses, and a course can have multiple students, which is represented through the Enrollment class using parallel arrays of student IDs and course codes. The Enrollment class depends on both CourseManager and StudentManager to validate students and courses before enrollment.
- The CourseManager and StudentManager classes are used to manage collections of courses and students respectively. Each manager class maintains and manages multiple instances of its corresponding entity through an array, representing a one-to-many relationship which means, one CourseManager manages up to 100 Course objects, and one StudentManager manages up to 500 Student objects.
- The SuggestionAPI class serves as a supporting utility used by CourseManager, StudentManager, and Enrollment. It provides caching and search suggestion features, storing recently accessed course codes and student IDs to improve the search experience within the system.

2.0.0 Documentation Formatting

To ensure clarity and consistency, all documentation regarding the Student Learning Management System (SLMS) must follow the following standardized format:

- Font: Times New Roman
- Font size: 11pt or 12pt
- Line spacing: 1.5
- Headings:
 - Main headings: Bold, 16 pt
 - Subheadings: Bold, 14 pt
- Page numbers included at the bottom right of pages
- Sections numbered logically (e.g., 1.0, 1.1, 1.2)
- Diagrams presented using standard UML notation
- Document format: .PDF or .DOCX

This ensures clarity and readability throughout the document while maintaining consistency in structure, terminology, and formatting. It also supports a professional academic presentation and improves overall comprehension for the reader.

3.0.0 Coding File Formatting

To ensure clean, consistent, and maintainable code development for the Student Learning Management System (SLMS), the following coding file formatting standards shall be applied:

- One class per file shall be implemented. The file name must exactly match the class name.
- Indentation shall use four spaces only. The use of tab characters is strictly prohibited.
- Opening braces must be written on the same line as the class, method, or control structure declaration.
- The maximum line length shall not exceed 100 characters to improve readability.
- Each method must include a brief comment describing its purpose and functionality.
- Meaningful and descriptive names must be used for all variables, methods, classes, and constants. Abbreviations should be avoided unless commonly understood.
- Consistent formatting shall be maintained throughout the entire project source code.

These standards improve readability, maintainability, collaboration efficiency, and overall consistency of the source code.

4.0.0 Naming Conventions

To ensure consistency, readability, and maintainability of the Student Learning Management System (SLMS), standardized naming conventions shall be applied throughout the project.

4.1.0 Class Naming

1. Class names shall use PascalCase.
2. Class names must represent nouns.
3. Each class name must match its file name exactly.

Examples from SLMS:

- Course
- Student
- Enrollment
- CourseManager
- StudentManager

4.2.0 Attribute Naming

1. Attributes shall use camelCase.
2. Attribute names must describe the stored data clearly.
3. All attributes shall be declared private to enforce encapsulation.

Examples:

- courseName
- courseCode
- creditHour
- courseSummary
- msTeamsLink
- studentId
- firstName
- lastName

- email
- phoneNum
- courseArray
- studentArray
- courseCount
- studentCount

4.3.0 Method Naming

1. Methods shall use camelCase.
2. Method names must begin with a verb.
3. Getter and setter methods must follow standard accessor naming format.

Examples:

- getCourseName()
- setCourseName(String name)
- addCourse(Course course)
- searchStudent(String studentId)
- deleteCourse(String courseCode)
- viewAllStudents()
- displayCourse()

4.4.0 Constant Naming

- Constants shall use UPPER_CASE with underscores.
- Constants must be declared as static and final (if Java is used).

Example:

- MAX_COURSE_LIMIT
- MAX_STUDENT_LIMIT

4.5.0 Vocabulary Dictionary

To ensure a shared understanding among developers and stakeholders, the following standardized terminology shall be used:

Term	Definition
Student	A registered learner enrolled in one or more courses
Lecturer	A user responsible for managing course and student information
System Administrator	A user with full system access and management authority
Course	A subject offered within the SLMS containing course details
Enrollment	The association between a student and a course
Course Manager	A controller class responsible for managing course operations
Student Manager	A controller class responsible for managing student operations
Course Code	A unique identifier assigned to each course
Student ID	A unique identifier assigned to each student
Credit Hour	The academic credit value assigned to a course
MS Teams Link	Online meeting link associated with a course
Course Profile	Complete details of a course
Student Profile	Complete details of a student

These naming conventions and standardized vocabulary reduce ambiguity, improve collaboration efficiency and maintain consistency across the system.

5.0.0 Function and Methods Modularity

The Student Learning Management System (SLMS) shall follow modular programming principles to ensure scalability, maintainability, and clean system architecture.

5.1.0 Encapsulation Principle

1. All class attributes shall be declared as private.
2. Access to attributes shall be controlled using getter and setter methods.

This ensures:

- Data protection
- Controlled modification
- Improved security
- Easier debugging

5.2.0 Single Responsibility Principle

Each method shall perform only one specific task.

Examples:

- `addCourse()` – adds a new course only
- `searchCourse()` – searches for a course only
- `editStudent()` – updates student details only
- `viewAllCourses()` – displays course list only

Methods must not combine multiple responsibilities such as validation, storage and display logic in one function.

5.3.0 Getter and Setter Methods

Getter Methods

- Used to retrieve attribute values.
- Must not modify object state.

Examples:

- getId() • getId() • getId()
- getId()
- getId()
- getId()

Setter Methods

- Used to modify attribute values.
- Must include input validation where necessary.

Examples:

- setEmail(String email)
- setCreditHour(int hours)

5.4.0 Method Size and Complexity Control

To maintain readability:

- Each method should not exceed 30–40 lines of code.
- Nested control structures shall be minimized.
- Repeated logic must be extracted into separate helper methods.

5.5.0 Separation of Responsibilities

The system shall separate responsibilities into different classes:

Entity Classes:

- Course
- Student
- Enrollment

Management Classes:

- CourseManager
- StudentManager

Entity classes store data and basic behaviors while Manager classes handle system-level operations such as adding, editing, searching, and deleting.

This structure improves modularity and prevents tightly coupled code.

5.6.0 Reusability and Maintainability

Common logic such as searching or validation must be reusable and placed in dedicated methods instead of duplicating code across multiple classes.

This modular structure ensures:

- Easier debugging
- Reduced redundancy
- Improved scalability
- Cleaner code maintenance

6.0.0 Continuous Integration Rules

The Student Learning Management System (SLMS) project shall use GitHub as the platform for version control and continuous integration management. Continuous integration practices ensure stable development, controlled collaboration, and high-quality source code throughout the project lifecycle.

6.1.0 Branching Strategy

The following branching strategy shall be implemented to maintain structured development:

- The main branch shall be named main.
- Feature branches shall be created from the main branch for each new functionality.
- Feature branches shall follow the naming format:
 - o feature/course-management
 - o feature/student-management
 - o feature/profile-management
- Feature branches must be merged back into the main branch only after approval.

6.2.0 Commit Message Format

All commit messages must follow a clear and standardized format to improve traceability and project documentation. Examples include:

- feat: add course profile creation function
- feat: implement student search feature
- fix: correct student id validation error
- refactor: improve course manager structure

Commit messages must be concise, descriptive, and written in lowercase after the prefix.

6.3.0 Merge Rules

The following rules shall be enforced before merging any branch into the main branch:

- A pull request must be created before merging.
- The code must compile successfully without errors.

- The code must comply with the defined naming conventions and formatting standards.
- Peer review approval is required before merging.

These continuous integration rules ensure structured collaboration, reduce integration conflicts, and maintain high software quality standards.

7.0.0 System Main Menu

The Main Menu serves as the entry point of the cherrySLMS system. It provides users with a list of core system functionalities, allowing them to navigate to different management modules. The available options include managing courses, managing students, managing enrollments, and exiting the system. Each option directs the user to the respective module for further operations. This menu ensures a simple and organized navigation structure for users.

```
===== Welcome to cherrySLMS! =====  
  
1. Manage Courses  
2. Manage Students  
3. Manage Enrollments  
4. Exit  
  
Choose option: -
```

Figure 7.1: SLMS Main Menu

7.0.1 Manage Courses Function

To access the Course Management Menu, the user must enter the option “1” from the Main Menu.

```
===== Welcome to cherrySLMS! =====  
  
1. Manage Courses  
2. Manage Students  
3. Manage Enrollments  
4. Exit  
  
Choose option: 1
```

Figure 7.2: User chooses option “1” from the SLMS Main Menu

7.0.2 Manage Students Function

To access the Student Management Menu, the user must enter the option “2” from the Main Menu.

```
===== Welcome to cherrySLMS! =====

1. Manage Courses
2. Manage Students
3. Manage Enrollments
4. Exit

Choose option: 2
```

Figure 7.3: User chooses option “2” from the SLMS Main Menu

7.0.3 Manage Enrollments Function

To access the Enrollment Management Menu, the user must enter the option “3” from the Main Menu.

```
===== Welcome to cherrySLMS! =====

1. Manage Courses
2. Manage Students
3. Manage Enrollments
4. Exit

Choose option: 3
```

Figure 7.4: User chooses option “3” from the SLMS Main Menu

7.0.4 Exit System

To exit the system, the user must enter the option “4” from the Main Menu.

```
===== Welcome to cherrySLMS! =====

1. Manage Courses
2. Manage Students
3. Manage Enrollments
4. Exit

Choose option: 4
Exiting system...
```

Figure 7.5: User chooses option “4” from the SLMS Main Menu

7.0.5 Input Validation

The system performs input validation to ensure that the user enters a valid option.

Case 1: User does not enter an integer.

```
===== Welcome to cherrySLMS! =====  
  
1. Manage Courses  
2. Manage Students  
3. Manage Enrollments  
4. Exit  
  
Choose option: p  
Invalid input! Please enter an integer.
```

Figure 7.6: User enters an invalid input

Case 2: User enters a number outside 1–4.

```
===== Welcome to cherrySLMS! =====  
  
1. Manage Courses  
2. Manage Students  
3. Manage Enrollments  
4. Exit  
  
Choose option: 6  
Please enter a number between 1 and 4.
```

Figure 7.7: User enters an invalid number

Case 3: User enters a valid option.

```
===== Welcome to cherrySLMS! =====
```

1. Manage Courses
2. Manage Students
3. Manage Enrollments
4. Exit

```
Choose option: 2
```

```
=== Student Management Menu ===
```

1. Add Student
2. Search Student
3. Edit Student
4. Delete Student
5. View All Students
6. Back

```
Choose option: _
```

Figure 7.8: User enters a valid input

8.0.0 Course Management Menu

The course management menu allows the user to manage course records within the system. It presents a list of options such as adding a new course, searching for existing courses, editing course details, deleting courses, and viewing all available courses. Each option directs the user to the corresponding function. Additionally, a “Back” option is provided to return to the SLMS main menu, ensuring easy navigation within the system.

```
=== Course Management Menu ===  
  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
6. Back  
  
Choose option: |
```

Figure 8.1: Course Management Menu

8.0.1 Add Course Function

To add a new course, the user must enter the option “1” from the Course Management Menu.

```
=== Course Management Menu ===  
  
1. Add Course  
2. Search Course  
3. Edit Course  
4. Delete Course  
5. View All Courses  
6. Back  
  
Choose option: 1
```

Figure 8.2: User chooses option “1” from the Course Management Menu

Then, the user is prompted to enter values for the course attributes such as course name, course code, credit hour, course summary and MS teams link. Input validation check is performed for credit hour, to check whether user enters a valid integer or not.

Case 1: User does not enter an integer for credit hour.

```
Enter Credit Hour: h
Invalid input! Please enter an integer.
```

Figure 8.3: User enters invalid input for credit hour

Case 2: User enters a negative integer for credit hour.

```
Enter Credit Hour: -5
Credit hour must be a positive integer.
```

Figure 8.4: User enters negative integer for credit hour

Case 3: User enters a valid integer for credit hour.

```
Enter Course Code: CSEB5223
Enter Course Name: Software Construction & Methods
Enter Credit Hour: 3
Enter Course Summary: This course is designed to provide basic
Enter MS Teams Link: https://teams.microsoft.com/cseb5223

Course added successfully.
```

Figure 8.5: User enters valid input for credit hour, so course is successfully added

Before adding a new course, the system will validate the array capacity to prevent any out-of-bounds errors.

Case 1: The course list is full and user is unable to add a new course.

```
Course list is full. Cannot add new course.
```

Figure 8.6: User is unable to add a course because course list is full

Case 2: The new course is added successfully.

```
Enter Course Code: CSEB5223
Enter Course Name: Software Construction & Methods
Enter Credit Hour: 3
Enter Course Summary: This course is designed to provide basic
Enter MS Teams Link: https://teams.microsoft.com/cseb5223

Course added successfully.
```

Figure 8.7: Course is successfully added

8.0.2 Search Course Function

To search for a course, the user must enter the option “2” from the Course Management Menu. Then, the user will be prompted to enter the course code of the course they want to search (case-insensitive). If the course is found, the corresponding course details will be displayed.

```
=== Course Management Menu ===

1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Back

Choose option: 2
Enter course code to search: CSEB5223
```

Figure 8.8: User chooses option “2” from the Course Management Menu

Case 1: Course is found.

```
=== Course Management Menu ===

1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Back

Choose option: 2

--- Recent Searches ---
> No recent searches available.
Enter course code: CSEB5223

Did you mean:
- CSEB5223
Confirm course code: CSEB5223

Course found:

Course Name: Software Construction & Methods
Course Code: CSEB5223
Credit Hour: 3
Summary: This course is designed to provide basic
MS Teams Link: https://teams.microsoft.com/cseb5223
```

Figure 8.9: User enters course code that is in the system, so the course is found

Case 2: Course is not found.

```
=== Course Management Menu ===

1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Back

Choose option: 2

--- Recent Searches ---
> CSEB5233
> CSEB5223
Enter course code: cseb5333

Did you mean:
- No matches found.
Confirm course code: cseb5333

Course not found.
```

Figure 8.10: User enters course code that is not in the system, so the course is not found

8.0.3 Edit Course Function

To edit a course details, the user must enter the option “3” from the Course Management Menu. Then, the user will be prompted to enter the course code of the course they want to edit (case-insensitive).

```

=== Course Management Menu ===

1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Back

Choose option: 3

--- Recent Searches ---
> CSEB5333
> CSEB5233
> CSEB5223
Enter course code to edit: CSEB5223

Course found.

Course Name: Software Construction & Methods
Course Code: CSEB5223
Credit Hour: 3
Summary: This course is designed to provide basic
MS Teams Link: https://teams.microsoft.com/cseb5223

```

Figure 8.11: User chooses option “3” from the Course Management Menu

After the course code is entered, the course details will be displayed and the user will be asked again for confirmation (case-insensitive) if they want to edit the course selected.

Case 1: Course is found and edited successfully. Updated course details will be displayed.

```

Course found.

Course Name: Software Construction & Methods
Course Code: CSEB5223
Credit Hour: 3
Summary: This course is designed to provide basic
MS Teams Link: https://teams.microsoft.com/cseb5223

Are you sure you want to edit this course? (Y/N): Y

Enter New Course Name: SCM
Enter New Credit Hour: 2
Enter New Summary: This course is designd to provide a basic understanding
Enter New MS Teams Link: https://teams.microsoft.com/scm

Course updated successfully.
Updated details:

Course Name: SCM
Course Code: CSEB5223
Credit Hour: 2
Summary: This course is designd to provide a basic understanding
MS Teams Link: https://teams.microsoft.com/scm

```

Figure 8.12: The system successfully finds the course, allows the user to edit it, and then displays the updated course details

Case 2: Course is found but edit is cancelled.

```
Choose option: 3

--- Recent Searches ---
> CSEB5223
> CSEB5333
> CSEB5233
Enter course code to edit: CSEB5223

Course found.

Course Name: SCM
Course Code: CSEB5223
Credit Hour: 2
Summary: This course is designd to provide a basic understanding
MS Teams Link: https://teams.microsoft.com/scm

Are you sure you want to edit this course? (Y/N): N
Edit cancelled.
```

Figure 8.13 When the course is found, the user chooses to cancel the edit, and the course details remain unchanged

8.0.4 Delete Course Function

To delete a course, the user must enter the option “4” from the Course Management Menu. Then, the user will be prompted to enter the course code of the course they want to delete (case-insensitive).

```
=== Course Management Menu ===

1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Back

Choose option: 4
Enter course code to delete: CSEB4113
```

Figure 8.14: User chooses option “4” from the Course Management Menu

After the course code is entered, the course details will be displayed and the user will be asked again for confirmation (case-insensitive) if they want to delete the course selected.

Case 1: Course is found and deleted successfully. Updated course list will be displayed without deleted course.

```
Enter course code to delete: CSEB4113

Course found.

Course Name: Software Quality
Course Code: CSEB4113
Credit Hour: 3
Summary: Learn about upholding a software's quality.
MS Teams Link: https://teams.microsoft.com/cseb4113

Are you sure you want to delete this course? (Y/N): Y

Course deleted successfully!
Updated course list:

Course Name: SCM
Course Code: CSEB5223
Credit Hour: 2
Summary: This course is designed to provide a basic understanding
MS Teams Link: https://teams.microsoft.com/scm
-----
```

Figure 8.15: The system successfully finds and deletes the course, and the updated course list is displayed without the deleted course

Case 2: Course is found but delete is cancelled.

```
Enter course code to delete: CSEB4113

Course found.

Course Name: Software Quality
Course Code: CSEB4113
Credit Hour: 3
Summary: Learn about upholding a software's quality.
MS Teams Link: https://teams.microsoft.com/cseb4113

Are you sure you want to delete this course? (Y/N): N
Deletion cancelled.
```

Figure 8.16: The system successfully finds the course, but the user decides to cancel the deletion, so no changes are made to the course list

8.0.5 View All Courses Function

To view all courses, the user must enter the option “5” from the Course Management Menu.

```
=== Course Management Menu ===

1. Add Course
2. Search Course
3. Edit Course
4. Delete Course
5. View All Courses
6. Back

Choose option: 5
```

Figure 8.17: User chooses option “5” from the Course Management Menu

```
Choose option: 5

=== All Courses ===

Course Name: SCM
Course Code: CSEB5223
Credit Hour: 2
Summary: This course is designd to provide a basic understanding
MS Teams Link: https://teams.microsoft.com/scm
-----
Course Name: Software Quality
Course Code: CSEB4113
Credit Hour: 3
Summary: Learn about upholding a software's quality.
MS Teams Link: https://teams.microsoft.com/cseb4113
-----
```

Figure 8.18: The system retrieves and displays all available courses to the user

8.1.0 Analysis of Software Construction Best Practices and Challenges

8.1.1 Software Construction Best Practices

1. Encapsulation

Used private access modifiers for attributes in the Course class, ensuring data integrity through public getter and setter methods.

2. Modular Design

To simplify debugging and maintenance, the system is divided into three distinct files:

- 1) Course.java (Model)
- 2) CourseManager.java (Controller/Logic)
- 3) SLMS.java (View/Interface)

3. Defensive Programming

i) Boundary Checking

Implemented a manual check in addCourse to prevent ArrayIndexOutOfBoundsException when the array reaches its 100-item limit.

ii) Input Validation

Used try-catch blocks and Integer.parseInt() to handle NumberFormatException, preventing program from crashing if a user enters non-numeric data for credit hours.

4. Data Integrity during Deletion

When a course is deleted, the program manually shifts subsequent array elements to the left and nullifies the last index to maintain a contiguous list.

8.1.2 Challenges

1. Managing course storage using arrays

Arrays have a fixed size, so adding too many courses may cause an array out-of-bound error. To prevent this, a `courseCount` variable is used to track the number of stored courses and ensure new courses are only added if there is space in the array.

2. Implementing the linear search function

The system must go through the array to find a course that matches the entered course code. The challenge was making sure the program correctly compares the input with the stored course codes and returns the correct result.

3. Handling unsuccessful searches

If the entered course code does not exist, the program must handle it properly. The system returns null and displays a message such as “Course not found” to inform the user.

4. Input validation for credit hour

Users may enter invalid inputs such as letters instead of numbers. A try-catch block is used to ensure that only valid integer values are accepted for the credit hour.

5. Displaying course information clearly

Course information needs to be shown in a clear and organized way. The `displayCourse()` method is used to display all course details in a consistent format so users can read the information easily.

9.0.0 Student Management Menu

The student management menu allows the user to manage student records within the system. It presents a list of options such as adding a new student, searching for existing students, editing student details, deleting students, and viewing all students. Each option directs the user to the corresponding function. Additionally, a “Back” option is provided to return to the SLMS main menu, ensuring easy navigation within the system.

```
=== Student Management Menu ===  
  
1. Add Student  
2. Search Student  
3. Edit Student  
4. Delete Student  
5. View All Students  
6. Back  
  
Choose option: ||
```

Figure 9.1: Student Management Menu

9.0.1 Add Student Function

To add a new student, the user must enter the option “1” from the Student Management Menu.

```
=== Student Management Menu ===  
  
1. Add Student  
2. Search Student  
3. Edit Student  
4. Delete Student  
5. View All Students  
6. Back  
  
Choose option: 1
```

Figure 9.2: User chooses option “1” from the Student Management Menu

Then, the user is prompted to enter values for the student attributes such as first name, last name, student ID, email and phone number. Input validation checks are performed for email and phone number, to check whether user enters a valid format or not.

Case 1: User does not enter a valid email address.

```
Enter Email: aa
Invalid email format! Please try again.
```

Figure 9.3: User enters invalid input for email

Case 2: User does not enter a valid phone number.

```
Enter Phone Number: 876
Invalid phone number! Please try again.
```

Figure 9.4: User enters invalid input for phone number

Case 3: User enters a valid email address and phone number.

```
Enter First Name: Aimi
Enter Last Name: Syazwani
Enter Student ID: BSW01085441
Enter Email: aimi@gmail.com
Enter Phone Number: 0192345678

Student added successfully.
```

Figure 9.5: User enters valid input for email and phone number

Before adding a new student, the system will validate the array capacity to prevent any out-of-bounds errors.

Case 1: The student list is full and user is unable to add a new student.

```
Student list is full. Cannot add new student
```

Figure 9.6: User is unable to add a student because student list is full

Case 2: The new course is added successfully.

```
Enter First Name: Aimi
Enter Last Name: Syazwani
Enter Student ID: BSW01085441
Enter Email: aimi@gmail.com
Enter Phone Number: 0192345678

Student added successfully.
```

Figure 9.7: Student is successfully added

9.0.2 Search Student Function

To search for a student, the user must enter the option “2” from the Student Management Menu. Then, the user will be prompted to enter the student ID of the student they want to search (case-insensitive). If the student is found, the corresponding student details will be displayed.

```
=== Student Management Menu ===

1. Add Student
2. Search Student
3. Edit Student
4. Delete Student
5. View All Students
6. Back

Choose option: 2
Enter student ID to search: BSW01085441
```

Figure 9.8: User chooses option “2” from the Student Management Menu

Case 1: Student is found.

```
=== Student Management Menu ===

1. Add Student
2. Search Student
3. Edit Student
4. Delete Student
5. View All Students
6. Back

Choose option: 2

--- Recent Searches ---
> No recent searches available.
Enter student ID: BSW01085441

Did you mean:
- BSW01085441
Confirm student ID: BSW01085441

Student found:

Student ID: BSW01085441
Name: Aimi Syazwani
Email: aimi@gmail.com
Phone: 0192345678
```

Figure 9.9: User enters student id that is in the system, so the student is found

Case 2: Student is not found.

```
Enter student ID: bsw01085444

Did you mean:
- No matches found.
Confirm student ID: bsw01085444

Student not found.
```

Figure 9.10: User enters student id that is not in the system, so the student is not found

9.0.3 Edit Student Function

To edit a student's details, the user must enter the option "3" from the Student Management Menu. Then, the user will be prompted to enter the student ID of the student they want to edit (case-insensitive).

```
=== Student Management Menu ===

1. Add Student
2. Search Student
3. Edit Student
4. Delete Student
5. View All Students
6. Back

Choose option: 3
Enter student ID to edit: BSW01085261
```

Figure 9.11: User chooses option “3” from the Student Management Menu

After the student ID is entered, the student details will be displayed and the user will be asked again for confirmation (case-insensitive) if they want to edit the student selected.

Case 1: Student is found and edited successfully. Updated student details will be displayed.

```
Enter student ID to edit: BSW01085247

Student found.

Student ID: BSW01085247
Name: Nur Sahira
Email: sahirah@gmail.com
Phone: 0112345678

Are you sure you want to edit this student? (Y/N): y

Enter New First Name: Nurul
Enter New Last Name: Sahirah
Enter New Email: sahirah@gmail.com
Enter New Phone Number: 0123456789

Student updated successfully.
Updated details:

Student ID: BSW01085247
Name: Nurul Sahirah
Email: sahirah@gmail.com
Phone: 0123456789
```

Figure 9.12: The system successfully finds the student, allows the user to edit it, and then displays the updated student details

Case 2: Student is found but edit is cancelled.

```
Enter student ID to edit: BSW01085247

Student found.

Student ID: BSW01085247
Name: Nur Sahira
Email: sahira@gmail.com
Phone: 0112345678

Are you sure you want to edit this student? (Y/N): n
Edit cancelled.
```

Figure 9.13 When the student is found, the user chooses to cancel the edit, and the student details remain unchanged

9.0.4 Delete Student Function

To delete a student, the user must enter the option “4” from the Student Management Menu. Then, the user will be prompted to enter the student ID of the student they want to delete (case-insensitive).

```
=== Student Management Menu ===

1. Add Student
2. Search Student
3. Edit Student
4. Delete Student
5. View All Students
6. Back

Choose option: 4
Enter student ID to delete: BSW01085247
```

Figure 9.14: User chooses option “4” from the Student Management Menu

After the student ID is entered, the student details will be displayed and the user will be asked again for confirmation (case-insensitive) if they want to delete the student selected.

Case 1: Student is found and deleted successfully. Updated student list will be displayed without deleted student.

```
Enter student ID to delete: BSW01085441

Student found.

Student ID: BSW01085441
Name: Aimi Syazwani
Email: aimi@gmail.com
Phone: 0192345678

Are you sure you want to delete this student? (Y/N): Y

Student deleted successfully!
Updated student list:

Student ID: BSW01085247
Name: Nurul Sahirah
Email: sahirah@gmail.com
Phone: 0123456789
-----
```

Figure 9.15: The system successfully finds and deletes the student, and the updated student list is displayed without the deleted course

Case 2: Student is found but delete is cancelled.

```
Enter student ID to delete: BSW01085441

Student found.

Student ID: BSW01085441
Name: Aimi Syazwani
Email: aimi@gmail.com
Phone: 0192345678

Are you sure you want to delete this student? (Y/N): n
Deletion cancelled.
```

Figure 9.16: The system successfully finds the student, but the user decides to cancel the deletion, so no changes are made to the student list

9.0.5 View All Students Function

To view all students, the user must enter the option “5” from the Student Management Menu.

```
=== Student Management Menu ===  
  
1. Add Student  
2. Search Student  
3. Edit Student  
4. Delete Student  
5. View All Students  
6. Back  
  
Choose option: 5
```

Figure 9.17: User chooses option “5” from the Student Management Menu

```
=== All Students ===  
  
Student ID: BSW01085441  
Name: Aimi Syazwani  
Email: aimi@gmail.com  
Phone: 0192345678  
-----  
Student ID: BSW01085247  
Name: Nurul Sahirah  
Email: sahirah@gmail.com  
Phone: 0123456789  
-----
```

Figure 9.18: The system retrieves and displays all available students to the user

9.1.0 Analysis of Software Construction Best Practices and Challenges

9.1.1 Software Construction Best Practices

1. Encapsulation

Used private access modifiers for attributes in the Student class, ensuring data integrity through public getter and setter methods.

2. Modular Design

The system is split into three files to make it easier to manage: Student.java stores student data, StudentManager.java handles adding, searching, editing, and deleting students, and SLMS.java is the user interface.

3. Input Validation

Validation uses regular expressions (regex) to make sure input is correct. Emails must be in a valid format, and phone numbers must contain only digits and have the right length. This prevents invalid data from being stored.

4. Error Handling

The program includes checks to handle possible errors, such as stopping new students from being added when the array is full and showing a “Student not found” message when no match is found. This helps the program run smoothly and gives clear feedback to the user.

5. Reusability

The searchStudent() method is reused in multiple operations such as edit and delete. This avoids repeating the same code and makes the program more efficient and easier to maintain.

6. Clear Output Formatting

The displayStudent() method is used to present student details in a consistent and organized format. This improves readability and ensures users can easily understand the displayed information.

9.1.2 Challenges

1. Managing course storage using arrays

Arrays have a fixed size, which limits the number of students that can be stored. A `studentCount` variable is used to control insertion and prevent overflow.

2. Implementing the linear search function

The system uses a linear search to locate students by ID. The challenge is ensuring accurate comparison and efficient traversal of stored data.

3. Handling unsuccessful searches

If a student ID is not found, the system must return null and display an appropriate message such as “Student not found” without causing errors.

4. Validating user input formats

Ensuring correct formats for email and phone number using regex can be complex, especially when handling different possible input variations.

5. Maintaining data consistency during edit and delete urse information clearly

Updating and removing student records requires careful handling to ensure the correct record is modified or deleted without affecting other data.

6. Displaying student information clearly

Student details must be presented in a clear and consistent format. The `displayStudent`.

10.0.0 Enrollment Management Menu

Based on the previous version of the main system menu, the new version features a “Manage Enrollments” menu to manage course and student relationships.

```
===== Welcome to cherrySLMS! =====  
  
1. Manage Courses  
2. Manage Students  
3. Manage Enrollments  
4. Exit
```

Figure 10.1: SLMS Main Menu

It provides options such as adding a course to a student, adding a student to a course, searching for a student’s course using their student ID, and listing all courses a student is enrolled in. Users can also find a specific student within a course using the course code and view all students enrolled in a particular course. Each option directs the user to the corresponding function. Additionally, a “Back” option is included to return to the SLMS main menu, ensuring smooth and convenient navigation.

```
=== Enrollment Management Menu ===  
  
1. Add a course to a student  
2. Add a student to a course  
3. Find a student's course by student ID  
4. List all courses enrolled by a student  
5. Find a student in a course by course code  
6. List all students enrolled in a course  
7. Back
```

Figure 10.2: Enrollment Management Menu

10.0.1 Add a Course to a Student Function

To add a new course to a student, the user must enter the option “1” from the Enrollment Management Menu. Then, the user is prompted for the student ID (case-insensitive) of the student selected and the course code (case-insensitive) of the course that they wish to add to the student.

```
=== Enrollment Management Menu ===

1. Add a course to a student
2. Add a student to a course
3. Find a student's course by student ID
4. List all courses enrolled by a student
5. Find a student in a course by course code
6. List all students enrolled in a course
7. Back

Choose option: 1
```

Figure 10.3: User chooses option “1” from the Enrollment Management Menu

Case 1: Course enrollment is no longer available because enrollment list is full.

```
=== Enrollment Management Menu ===

1. Add a course to a student
2. Add a student to a course
3. Find a student's course by student ID
4. List all courses enrolled by a student
5. Find a student in a course by course code
6. List all students enrolled in a course
7. Back

Choose option: 1

Enrollment list is full.
```

Figure 10.4: User is unable to add a course to student because enrollment list is full

Case 2: Student ID does not exist/not found.

```
Enter Student ID: BSW01084321

Student not found.
```

Figure 10.5: User enters student ID that is not in the system, so the student is not found

Case 3: Course code does not exist/not found.

```
Enter Student ID: BSW01085441
Enter Course Code: CCSB4133

Course not found.
```

Figure 10.6: User enters course code that is not in the system, so the course is not found

Case 4: Student is already enrolled in the selected course.

```
Enter Student ID: BSW01085441
Enter Course Code: CSEB5223

Student is already enrolled in this course.
```

Figure 10.7: User enters enrollment details that are already in the system

Case 5: The course is added successfully to the student.

```
Enter Student ID: BSW01085247
Enter Course Code: CSEB5223

Course successfully added to student.
```

Figure 10.8: User enters student ID and course code where course is successfully added to the student

10.0.2 Add a Student to a Course Function

To add a new student to a course, the user must enter the option “2” from the Enrollment Management Menu. Then, the user is prompted for the course code (case-insensitive) of the course selected and the student ID (case-insensitive) of the student that they wish to add to the course.

```
=== Enrollment Management Menu ===

1. Add a course to a student
2. Add a student to a course
3. Find a student's course by student ID
4. List all courses enrolled by a student
5. Find a student in a course by course code
6. List all students enrolled in a course
7. Back

Choose option: 2
```

Figure 10.9: User chooses option “2” from the Enrollment Management Menu

Case 1: Student enrollment is no longer available because enrollment list is full.

```
=== Enrollment Management Menu ===

1. Add a course to a student
2. Add a student to a course
3. Find a student's course by student ID
4. List all courses enrolled by a student
5. Find a student in a course by course code
6. List all students enrolled in a course
7. Back

Choose option: 2

Enrollment list is full.
```

Figure 10.10: User is unable to add a student to course because enrollment list is full

Case 2: Course code does not exist/not found.

```
Enter Course Code: CSEB6123

Course not found.
```

Figure 10.11: User enters course code that is not in the system, so the course is not found

Case 3: Student ID does not exist/not found.

```
Enter Course Code: CSEB5223
Enter Student ID: BSW01087667

Student not found.
```

Figure 10.12: User enters student ID that is not in the system, so the student is not found

Case 4: Student is already enrolled in the selected course.

```
Enter Course Code: CSEB5223
Enter Student ID: BSW01085247

Student is already enrolled in this course.
```

Figure 10.13: User enters enrollment details that are already in the system

Case 5: The student is added successfully to the course.

```
Enter Course Code: CCSB5113
Enter Student ID: BSW01085441

Student successfully added to course.
```

Figure 10.14: User enters course code and student ID where student is successfully added to the course

10.0.3 Find a Student's Course by Student ID Function

To find a student's course by their student ID, the user must enter the option "3" from the Enrollment Management Menu. Then, the user is prompted for the student ID (case-insensitive) of the student selected to search and one more time as confirmation before processing the request. The system implements a cache API to show the recent searches of student IDs saved within the system cache. The system will then display the course code of the earliest course that the student has been enrolled into.

```
=== Enrollment Management Menu ===

1. Add a course to a student
2. Add a student to a course
3. Find a student's course by student ID
4. List all courses enrolled by a student
5. Find a student in a course by course code
6. List all students enrolled in a course
7. Back

Choose option: 3
```

Figure 10.15: User chooses option “3” from the Enrollment Management Menu

Case 1: Student is enrolled in a course(s).

```
--- Recent Searches ---
> BSW01085247
Enter student ID: BSW01085441

Did you mean:
- BSW01085441
Confirm student ID: BSW01085441

Course found: CSEB5223
```

Figure 10.16: User enters student ID and the first ever course they have enrolled in appears

Case 2: Student is not enrolled in any course(s).

```
--- Recent Searches ---
> BSW01085441
> BSW01085247
Enter student ID: BSW01085261

Did you mean:
- BSW01085261
Confirm student ID: BSW01085261

No course found for this student.
```

Figure 10.17: User enters student ID where the student is not enrolled into any course

10.0.4 List all Courses enrolled by a Student Function

To list all courses enrolled by a student, the user must enter the option “4” from the Enrollment Management Menu. Then, the user is prompted for the student ID (case-insensitive) of the student selected to search and one more time as confirmation before processing the request. The system implements a cache API to show the recent searches of student IDs saved within the system cache. The system will then display the course codes of all the courses that the selected student is enrolled in.

```
=== Enrollment Management Menu ===

1. Add a course to a student
2. Add a student to a course
3. Find a student's course by student ID
4. List all courses enrolled by a student
5. Find a student in a course by course code
6. List all students enrolled in a course
7. Back

Choose option: 4
```

Figure 10.18: User chooses option “4” from the Enrollment Management Menu

Case 1: Student is enrolled in a course(s) and all courses are displayed.

```
--- Recent Searches ---
> BSW01085261
> BSW01085441
> BSW01085247
Enter student ID: BSW01085441

All courses enrolled by student BSW01085441:
> CSEB5223
> CCSB5113
```

Figure 10.19: User enters student ID and all courses they are enrolled in appears

Case 2: Student is not enrolled in any course(s).

```
--- Recent Searches ---
> BSW01085441
> BSW01085261
> BSW01085247
Enter student ID: BSW01085261

This student has no enrolled courses.
```

Figure 10.20: User enters student ID where the student is not enrolled in any course

10.0.5 Find a Student in a Course by Course Code Function

To list all courses enrolled by a student, the user must enter the option “5” from the Enrollment Management Menu. Then, the user is prompted for the course code (case-insensitive) of the course selected to search and one more time as confirmation before processing the request. The system implements a cache API to show the recent searches of course codes saved within the system cache. The system will then display the student ID of the earliest student that has enrolled in the course.

```
=== Enrollment Management Menu ===

1. Add a course to a student
2. Add a student to a course
3. Find a student's course by student ID
4. List all courses enrolled by a student
5. Find a student in a course by course code
6. List all students enrolled in a course
7. Back

Choose option: 5
```

Figure 10.21: User chooses option “5” from the Enrollment Management Menu

Case 1: Course has student(s) enrolled in it.

```
--- Recent Searches ---
> CSEB5223
> CSEB4113
> 1
> CSEB5123
Enter course code: CSEB5223

Did you mean:
- CSEB5223
Confirm course code: CSEB5223

Student found: BSW01085247
```

Figure 10.22: User enters course code and the first ever student that is enrolled in it appears

Case 2: Course does not have student(s) enrolled in it.

```
--- Recent Searches ---
> CSEB5223
> CSEB4113
> 1
> CSEB5123
Enter course code: CSEB4113

Did you mean:
- CSEB4113
Confirm course code: CSEB4113

No student found in this course.
```

Figure 10.23: User enters course code where no student is enrolled in it

10.0.6 List all Students enrolled in a Course Function

To list all courses enrolled by a student, the user must enter the option “6” from the Enrollment Management Menu. Then, the user is prompted for the course code (case-insensitive) of the course selected to search and one more time as confirmation before processing the request. The system implements a cache API to show the recent searches of course codes saved within the system cache. The system will then display the student IDs of all the students that are enrolled in the course.

```
=== Enrollment Management Menu ===

1. Add a course to a student
2. Add a student to a course
3. Find a student's course by student ID
4. List all courses enrolled by a student
5. Find a student in a course by course code
6. List all students enrolled in a course
7. Back

Choose option: 6
```

Figure 10.24: User chooses option “6” from the Enrollment Management Menu

Case 1: Course has student(s) enrolled in it.

```
--- Recent Searches ---
> CSEB4113
> CSEB5223
> 1
> CSEB5123
Enter course code: CSEB5223

All students enrolled in course CSEB5223:
> BSW01085247
> BSW01085441
```

Figure 10.25: User enters course code and all students that are enrolled in it appears

Case 2: Course does not have student(s) enrolled in it.

```
--- Recent Searches ---
> CSEB5223
> CSEB4113
> 1
> CSEB5123
Enter course code: CSEB4113

This course has no students.
```

Figure 10.26: User enters course code where there are no students enrolled in it

10.1.0 Implementation of the Suggestion API

The SuggestionAPI was implemented as a middleware layer to improve usability in the Student Learning Management System (SLMS). Its main purpose is to reduce repetitive typing, minimize user input errors, and improve search efficiency by storing previously entered student IDs and course codes.

Instead of relying only on manual input, the system keeps track of frequently used records through caching and recent search history. This allows users to receive suggestions when performing operations such as searching, editing, deleting, and managing enrollments.

The API is connected to multiple modules including Course Management, Student Management, and Enrollment Management through a single shared SuggestionAPI object created in SLMS.java.

10.1.1 Implementation of Caching in Course Management

In the Course Management module, the SuggestionAPI stores course codes using the `cacheCourse()` method. Whenever a valid course is added, searched, edited, deleted, or used in enrollment functions, the course code is saved into the `cachedCourses[]` array. This allows the system to reuse previously entered values instead of requiring users to manually remember and retype them.

```
// Add course code to cache
public void cacheCourse(String code) {
    if (code == null) {
        return;
    }

    code = code.toUpperCase();

    if (isCourseCached(code)) {
        return;
    }

    if (!isCourseCached(code) && courseCache < cachedCourses.length) {
        cachedCourses[courseCache++] = code;
    }
}
```

Figure 10.27: cacheCourse() method in SuggestionAPI.java

When the user selects functions such as Search Course, Edit Course, or Delete Course, the system first displays recent course searches using `showRecentCourseSearches()`. The user then enters a course code, and the `suggestCourseList()` method checks the input prefix against cached course codes using string matching with `startsWith()`.

```
// Suggest course codes based on input
public void suggestCourseList(String code) {
    if (code == null) {
        return;
    }
    code = code.toUpperCase();

    System.out.println("\nDid you mean:");
    boolean found = false;

    for (int i = 0; i < courseCache; i++) {
        String c = cachedCourses[i];

        if (c != null && c.startsWith(code)) {
            System.out.println(" - " + c);
            found = true;
        }
    }

    if (!found) {
        System.out.println(" - No matches found.");
    }
}
```

Figure 10.28: suggestCourseList() method in SuggestionAPI.java

For example, if the user enters *CSE*, the system will display matching suggestions such as *CSEB5123* under the “*Did you mean:*” section. This helps users quickly identify the correct course code and reduces typing mistakes. In addition, the system stores up to five recent course searches using `addRecentCourseSearch()`. This allows users to quickly refer back to previously accessed records without searching from the beginning again.

This implementation improves both efficiency and accuracy while keeping the menu interface simple, since all suggestion and caching logic is handled separately inside the middleware layer.

10.1.2 Implementation of Caching in Student Management

The same SuggestionAPI is also used in the Student Management module, showing how one middleware layer can support multiple data types without duplication of logic. Student IDs are stored using the `cacheStudent()` method inside the `cachedStudents[]` array.

```
// Add student ID to cache
public void cacheStudent(String id) {
    if (id == null) {
        return;
    }

    id = id.toUpperCase();

    if (isStudentCached(id)) {
        return;
    }

    if (!isStudentCached(id) && studentCache < cachedStudents.length) {
        cachedStudents[studentCache++] = id;
    }
}
```

Figure 10.29: cacheStudent() method in SuggestionAPI.java

Whenever users perform actions such as Search Student, Edit Student, Delete Student, or enrollment-related operations involving student IDs, the system records the input and stores it for future use. This ensures that once a student ID is entered anywhere in the system, it becomes available as a suggestion across all related modules.

Before searching or modifying a student record, the system displays recent student searches through `showRecentStudentSearches()`. When the user enters part of a student ID, the `suggestStudentList()` method compares the input with cached student IDs and displays possible matches.

```

// Suggest student IDs based on input
public void suggestStudentList(String id) {
    if (id == null) {
        return;
    }
    id = id.toUpperCase();

    System.out.println("\nDid you mean:");
    boolean found = false;

    for (int i = 0; i < studentCache; i++) {
        String s = cachedStudents[i];

        if (s != null && s.startsWith(id)) {
            System.out.println(" - " + s);
            found = true;
        }
    }

    if (!found) {
        System.out.println(" - No matches found.");
    }
}

```

Figure 10.30: *suggestStudentList()* method in *SuggestionAPI.java*

For example, entering *BSW* may display suggestions such as *BSW01085441*, helping users avoid input mistakes and improving data consistency.

This feature greatly improves user experience because users no longer need to memorize long alphanumeric Student IDs. Instead, the system provides visual guidance and reusable search history, reducing cognitive load and improving operational speed.

The integration of *SuggestionAPI* into both *manageStudents()* and *manageEnrollments()* demonstrates strong middleware design, where business logic is separated from the user interface while still supporting efficient interaction across the entire system.

10.2.0 Analysis of Software Construction Best Practices and Challenges

10.2.1 Software Construction Best Practices

1. Modularity

The system is designed using a modular approach by dividing functions into different classes such as Course, Student, CourseManager, StudentManager, Enrollment, and SuggestionAPI. Each class has its own role, where Course and Student store data, CourseManager and StudentManager handle operations, Enrollment manages relationships, and SuggestionAPI provides suggestions. This makes the code easier to read, maintain, and reuse. For example, changes in StudentManager do not affect CourseManager, showing good separation of tasks.

2. Encapsulation

Encapsulation is implemented by declaring class attributes as private and accessing them through getter and setter methods. This ensures that data is protected and can only be modified in controlled ways. By restricting direct access to variables, the system reduces the risk of unintended changes and improves data security. It also makes debugging easier since all data modifications go through defined methods.

3. Input Validation and Error Handling

The system includes input validation and error handling to ensure it runs properly. For example, integer inputs like credit hours are checked using try-catch blocks, while email and phone numbers are validated using patterns. The system also checks for duplicate entries and ensures that a student or course exists before performing any action. These checks help prevent errors and keep the system stable.

4. Use of Arrays for Data Storage

Arrays are used to store Course objects, Student objects, and enrollment relationships. This approach is simple and easy to implement, making it suitable for small-scale systems. Arrays also allow direct access to elements using indices, which improves performance. However, since arrays have fixed sizes, they are not flexible for larger systems.

5. Relationship Management

The relationship between students and courses is managed using parallel arrays in the Enrollment class. Student IDs and course codes are stored in corresponding positions, allowing the system to represent many-to-many relationships. Functions such as adding enrollments, listing courses for a student, and listing students in a course demonstrate effective relationship handling.

6. API and Caching Mechanism

The SuggestionAPI acts as a simple middleware API that caches student IDs and course codes. It provides suggestions to users before they input data, improving efficiency and user experience. This reduces repeated typing and improves overall system usability.

10.2.2 Challenges

1. Managing Relationships Using Arrays

Managing relationships using parallel arrays was challenging because it required maintaining consistency between multiple arrays. Searching for related data involved looping through arrays, which increased complexity and reduced efficiency compared to more advanced data structures.

2. Handling Duplicate Data

Preventing duplicate student IDs, course codes, and enrollments required multiple validation checks across different parts of the system. This made the logic more complex and sometimes repetitive, especially when ensuring data consistency.

3. Input Validation Complexity

Handling different types of input validation was challenging, as each input required a different approach. Integer validation used try-catch blocks, while email and phone

number validation required regular expressions. Ensuring all edge cases were handled correctly required careful implementation.

4. Fixed Array Size Limitation

Using fixed-size arrays limited the system's flexibility. There is a risk of reaching the maximum capacity, which would prevent adding new data. At the same time, unused array spaces may lead to inefficient memory usage.

5. Integration Between Classes

Integrating multiple classes such as CourseManager, StudentManager, Enrollment, and SuggestionAPI was challenging. Proper communication between these components required careful design, especially when passing objects between constructors.

6. Designing the Suggestion API

Developing the SuggestionAPI required ensuring that cached data remained accurate and up to date. Preventing duplicate entries and keeping the cache synchronized with the main data added complexity to the implementation.